PRACTICAL 10 (A)

Create a PWA with a Web App Manifest and implement Push Notifications

Aim

To create a Progressive Web Application (PWA) that includes an app name, icons, start URL, display mode, and push-notification functionality.

1. Create the Project

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Choose ASP.NET Core Empty Web App.
4. Click Next.
5. Enter the project name: PushNotification.
6. Click Create.

2. Create the Manifest File

1. In Solution Explorer, right-click the project and select Add → New Folder.
2. Name the folder wwwroot (if it does not already exist).
3. Right-click wwwroot → Add → New Item → JSON File.
4. Name the file manifest.json.
5. Enter the following JSON and save the file:

```
{
  "name": "My PWA Application",
  "short_name": "MyPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

3. Add Icons

1. Right-click wwwroot → Add → New Folder.
2. Name the folder icons.
3. Add icon1.png with dimensions 192 × 192 pixels.
4. Add icon2.png with dimensions 512 × 512 pixels.
5. Ensure the file names and paths match those specified in manifest.json.

4. Create the HTML Page

1. Right-click wwwroot → Add → New Item → HTML Page.
2. Name the file index.html.
3. Add the manifest link and app.js script as shown below, then save the file:

```
<!DOCTYPE html>
<html>
<head>
    <title>My PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
</head>
<body>
    <h1>My Progressive Web App</h1>
    <button onclick="showNotification()">Show Notification</button>
    <script src="app.js"></script>
</body>
</html>
```

5. Create the JavaScript File for Notifications

1. Right-click wwwroot → Add → New Item → JavaScript File.
2. Name the file app.js.
3. Enter the following code and save it:

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

function showNotification() {
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/icon1.png'
                });
            });
        }
    });
}
```

6. Create the Service Worker

1. Right-click wwwroot → Add → New Item → JavaScript File.
2. Name the file service-worker.js.
3. Enter the following code and save it:

```
self.addEventListener('install', event => {
    console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
    console.log('Service Worker Activated');
});
```

7. Configure Program.cs

Open Program.cs and ensure static files are enabled. The file should contain the following basic configuration:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Note: If the project template already includes any of these lines, avoid adding duplicates.

8. Run the Application

1. Run the project from Visual Studio.
2. Open the local HTTPS address shown by Visual Studio, for example https://localhost:7236.
3. The page should display “My Progressive Web App” and a “Show Notification” button.
4. If the install icon is available in the browser address bar, the PWA can be installed.

9. Install the PWA

1. Open the website in Google Chrome.
2. Click the Install icon in the address bar (if shown).
3. Confirm installation. The application should open like a standalone app.

10. Test Push Notification

1. Click the Show Notification button.
2. When the browser asks for notification permission, click Allow.
3. A notification should appear with the title “PWA Notification” and the message “Push Notification Working Successfully!”.

Expected Result

A PWA page opens in the browser, can be installed when supported, and displays a notification after the user grants permission.

Important Note

This practical demonstrates a local browser notification through the Service Worker. It does not implement server-based Web Push subscriptions or VAPID keys. Notifications generally require HTTPS or localhost, and the icon files must exist at the specified paths.